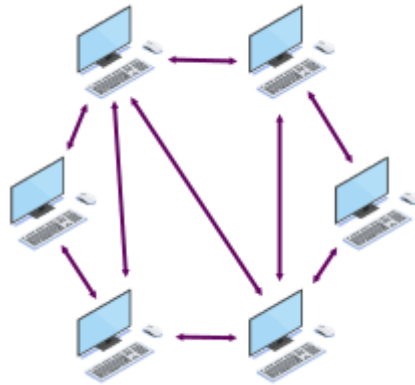


Modul 12

Teknologi P2P (*Peer-to-Peer*)

0. Pengantar Teknologi P2P



Teknologi P2P terdiri atas sekumpulan *nodes* yang terhubung secara langsung tanpa adanya suatu server yang menjadi perantara. Jadi, *node* bisa berfungsi sebagai *client* sekaligus berfungsi sebagai *server*. Beberapa kemungkinan penggunaan teknologi ini adalah sebagai berikut:

1. Berbagai file / *file sharing*
2. Aplikasi *Chat*
3. Aplikasi *games*

Materi praktikum ini akan membahas teknologi P2P dari berbagai segi.

1. Koneksi antar nodes

Program berikut ini merupakan program *chat* sederhana menggunakan Python. Dengan mempelajari program ini, bisa diketahui bagaimana cara koneksi antar *nodes* dilakukan serta bagaimana cara mengirimkan *message* antar *node* tersebut.

simple_chat.py

```
import socket
import threading
import sys

def terima_pesan(port_saya):
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    try:
        server_socket.bind(('0.0.0.0', port_saya))
        server_socket.listen(1)
        print(sys.stderr, f"\n[SERVER] Mendengarkan di port {port_saya}...")

        koneksi, alamat_peer = server_socket.accept()
        print(sys.stderr, f"\n[SERVER] Terhubung dengan peer: {alamat_peer}")

        while True:
            data = koneksi.recv(1024)
            if not data:
                print(sys.stderr, "\n[SERVER] Peer memutuskan koneksi.")
```

```

        break
    print(f"\n[Peer]: {data.decode('utf-8')}")

except Exception as e:
    print(sys.stderr, f"[SERVER] Error: {e}")
finally:
    server_socket.close()

def kirim_pesan(ip_tujuan, port_tujuan):
    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    print(sys.stderr, f"[CLIENT] Mencoba terhubung ke {ip_tujuan}:{port_tujuan}...")
    try:
        client_socket.connect((ip_tujuan, port_tujuan))
        print(sys.stderr, "[CLIENT] Sukses terhubung!")

        print("Silakan ketik pesan dan tekan Enter (Ketik 'keluar' untuk berhenti):")
        while True:
            pesan = input()
            if pesan.lower() == 'keluar':
                break
            client_socket.sendall(pesan.encode('utf-8'))

    except Exception as e:
        print(sys.stderr, f"[CLIENT] Gagal terhubung atau mengirim pesan: {e}")
    finally:
        client_socket.close()

if __name__ == "__main__":
    print("=== Praktikum Modul 12 - Sub 01 ===")

    port_lokal = int(input("Masukkan PORT LOKAL untuk server Anda (contoh: 5001): "))

    thread_server = threading.Thread(target=terima_pesan, args=(port_lokal,))
    thread_server.daemon = True
    thread_server.start()

    import time
    time.sleep(1)

    print("\n--- Konfigurasi Hubungan ke Peer Lain ---")
    ip_target = input("Masukkan IP TARGET (Peer tujuan, contoh: 192.168.1.10 atau localhost): ")
    port_target = int(input("Masukkan PORT TARGET (Port server peer tujuan): "))

    kirim_pesan(ip_target, port_target)

    print("\nProgram Selesai.")

```

```

1 import socket
1 import threading
2 import sys
3
4 def terima_pesan(port_saya):
5     server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
6     ---
7     try:
8         server_socket.bind(('0.0.0.0', port_saya))
9         server_socket.listen(1)
10        print(sys.stderr, f"\n[SERVER] Mendengarkan di port {port_saya}...")
11    ---
12        koneksi, alamat_peer = server_socket.accept()
13        print(sys.stderr, f"\n[SERVER] Terhubung dengan peer: {alamat_peer}")
14    ---
15        while True:
16            data = koneksi.recv(1024)
17            if not data:
18                print(sys.stderr, "\n[SERVER] Peer memutuskan koneksi.")
19                break
20            print(f"\n[Peer]: {data.decode('utf-8')}")
21    ---
22        except Exception as e:
23            print(sys.stderr, f"[SERVER] Error: {e}")
24        finally:
25            server_socket.close()
26
27 def kirim_pesan(ip_tujuan, port_tujuan):
28     client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
29     ---
30     print(sys.stderr, f"[CLIENT] Mencoba terhubung ke {ip_tujuan}:{port_tujuan}...")
31     try:
32         client_socket.connect((ip_tujuan, port_tujuan))
33         print(sys.stderr, "[CLIENT] Sukses terhubung!")
34     ---
35     print("Silakan ketik pesan dan tekan Enter (Ketik 'keluar' untuk berhenti):")
36     while True:
37         pesan = input()
38         if pesan.lower() == 'keluar':
39             break
40         client_socket.sendall(pesan.encode('utf-8'))
41    ---
42    except Exception as e:
43        print(sys.stderr, f"[CLIENT] Gagal terhubung atau mengirim pesan: {e}")
44    finally:
45        client_socket.close()
46
47 if __name__ == "__main__":
48     print("=== Praktikum Modul 12 - Sub 01 ===")
49     ---
50     port_lokal = int(input("Masukkan PORT LOKAL untuk server Anda (contoh: 5001): "))
51     ---
52     thread_server = threading.Thread(target=terima_pesan, args=(port_lokal,))
53     thread_server.daemon = True
54     thread_server.start()
55     ---
56     import time
57     time.sleep(1)
58     ---
59     print("\n--- Konfigurasi Hubungan ke Peer Lain ---")
60     ip_target = input("Masukkan IP TARGET (Peer tujuan, contoh: 192.168.1.10 atau localhost): ")
61     port_target = int(input("Masukkan PORT TARGET (Port server peer tujuan): "))
62     ---
63     kirim_pesan(ip_target, port_target)
64     ---
65     print("\nProgram Selesai.")

```

INSERT main bdp > simple_chat.py

Tugas:

1. Jalankan program tersebut sehingga muncul chat antar node satu dengan node lainnya. Sila bereksperimen dengan cara menjalankan program tersebut. *Capture* hasilnya dan masukkan dalam laporan.
2. Jelaskan bagian dalam program tersebut yang digunakan untuk:
 - a. Membuka port yang akan menerima dan mengirim pesan
 - b. Menerima pesan
 - c. Mengirim pesan

2. DHT (*Distributed Hash Table*)

DHT merupakan mekanisme yang biasanya digunakan oleh teknologi P2P untuk pencarian data tanpa adanya server yang menyimpan semua data. Jika pernah menggunakan Bittorrents, DHT merupakan salah satu komponen untuk pencarian dan koneksi ke node lainnya untuk keperluan mengambil file. Berikut adalah simulasi DHT menggunakan Python.

dht.py

```
import hashlib

def hitung_hash_8bit(teks):
    sha1 = hashlib.sha1(teks.encode('utf-8')).hexdigest()
    return int(sha1[-2:], 16)

class NodeP2P:
    def __init__(self, nama_node):
        self.nama = nama_node
        self.id = hitung_hash_8bit(nama_node)
        self.pen penyimpanan_lokal = {}
        print(f"Node '{self.nama}' berhasil dibuat dengan ID: {self.id}")

class LingkaranDHT:
    def __init__(self):
        self.daftar_node = []

    def tambah_node(self, node):
        self.daftar_node.append(node)
        self.daftar_node.sort(key=lambda x: x.id)

    def cari_node_terdekat(self, key_data):
        for node in self.daftar_node:
            if node.id >= key_data:
                return node
        return self.daftar_node[0]

    def simpan_data(self, nama_file, isi_konten):
        key_data = hitung_hash_8bit(nama_file)
        node_target = self.cari_node_terdekat(key_data)

        node_target.pen penyimpanan_lokal[key_data] = (nama_file, isi_konten)
        print(f"[SIMPAN] File '{nama_file}' (Key ID: {key_data}) disimpan di Node '{node_target.nama}' (Node ID: {node_target.id})")
```

```

def cari_data(self, nama_file):
    key_data = hitung_hash_8bit(nama_file)
    node_target = self.cari_node_terdekat(key_data)

    print(f"\n[PENCARIAN] Mencari file '{nama_file}' dengan Key ID: {key_data}...")
    print(f"[ROUTING] Request diarahkan ke Node terdekat: '{node_target.nama}' (Node ID: {node_target.id})")

    if key_data in node_target.penyimpanan_lokal:
        nama, konten = node_target.penyimpanan_lokal[key_data]
        print(f"[SUKSES] Data ditemukan! Isi konten: '{konten}'")
    else:
        print("[GAGAL] Data tidak ditemukan di jaringan.")

if __name__ == "__main__":
    print("=== Praktikum Modul 12 - Teknologi P2P - Simulasi Distributed Hash Table (DHT) ===")

    dht = LingkaranDHT()

    node_a = NodeP2P("Node A")
    node_b = NodeP2P("Node B")
    node_c = NodeP2P("Node C")

    dht.tambah_node(node_a)
    dht.tambah_node(node_b)
    dht.tambah_node(node_c)

    print("\nUrutan Node dalam Lingkaran DHT (Ring):")
    for n in dht.daftar_node:
        print(f" -> Node ID: {n.id} ({n.nama})")

    dht.simpan_data("tugas_jaringan.pdf", "Konten: Laporan Praktikum Modul 1")
    dht.simpan_data("foto_makrab.jpg", "Konten: Data biner gambar makrab angkatan")
    dht.simpan_data("source_code.py", "Konten: print('Hello P2P')")

    dht.cari_data("tugas_jaringan.pdf")
    dht.cari_data("source_code.py")
    dht.cari_data("praktikum.py")

```

```

1 import hashlib
2
3 def hitung_hash_8bit(teks):
4     sha1 = hashlib.sha1(teks.encode('utf-8')).hexdigest()
5     return int(sha1[-2:], 16)
6
7 class NodeP2P:
8     def __init__(self, nama_node):
9         self.nama = nama_node
10        self.id = hitung_hash_8bit(nama_node)
11        self.penimpanan_lokal = {}
12        print(f"Node '{self.nama}' berhasil dibuat dengan ID: {self.id}")
13
14 class LingkaranDHT:
15     def __init__(self):
16         self.daftar_node = []
17
18     def tambah_node(self, node):
19         self.daftar_node.append(node)
20         self.daftar_node.sort(key=lambda x: x.id)
21
22     def cari_node_terdekat(self, key_data):
23         for node in self.daftar_node:
24             if node.id >= key_data:
25                 return node
26         return self.daftar_node[0]
27
28     def simpan_data(self, nama_file, isi_konten):
29         key_data = hitung_hash_8bit(nama_file)
30         node_target = self.cari_node_terdekat(key_data)
31
32         node_target.penimpanan_lokal[key_data] = (nama_file, isi_konten)
33         print(f"[SIMPAN] File '{nama_file}' (Key ID: {key_data}) disimpan di Node '{node_target.nama}' (Node ID: {node_target.id})")
34
35     def cari_data(self, nama_file):
36         key_data = hitung_hash_8bit(nama_file)
37         node_target = self.cari_node_terdekat(key_data)
38
39         print(f"\n[PENCARIAN] Mencari file '{nama_file}' dengan Key ID: {key_data}...")
40         print(f"[ROUTING] Request diarahkan ke Node terdekat: '{node_target.nama}' (Node ID: {node_target.id})")
41
42         if key_data in node_target.penimpanan_lokal:
43             nama, konten = node_target.penimpanan_lokal[key_data]
44             print(f"[SUKSES] Data ditemukan! Isi konten: '{konten}'")
45         else:
46             print("[GAGAL] Data tidak ditemukan di jaringan.")
47
48 if __name__ == "__main__":
49     print("=== Praktikum Modul 12 - Teknologi P2P - Simulasi Distributed Hash Table (DHT) ===")
50
51     dht = LingkaranDHT()
52
53     node_a = NodeP2P("Node A")
54     node_b = NodeP2P("Node B")
55     node_c = NodeP2P("Node C")
56
57     dht.tambah_node(node_a)
58     dht.tambah_node(node_b)
59     dht.tambah_node(node_c)
60
61     print("\nUrutan Node dalam Lingkaran DHT (Ring):")
62     for n in dht.daftar_node:
63         print(f"    -> Node ID: {n.id} ({n.nama})")
64
65     dht.simpan_data("tugas_jaringan.pdf", "Konten: Laporan Praktikum Modul 1")
66     dht.simpan_data("foto_makrab.jpg", "Konten: Data biner gambar makrab angkatan")
67     dht.simpan_data("source_code.py", "Konten: print('Hello P2P')")
68
69     dht.cari_data("tugas_jaringan.pdf")
70     dht.cari_data("source_code.py")
71     dht.cari_data("praktikum.py")

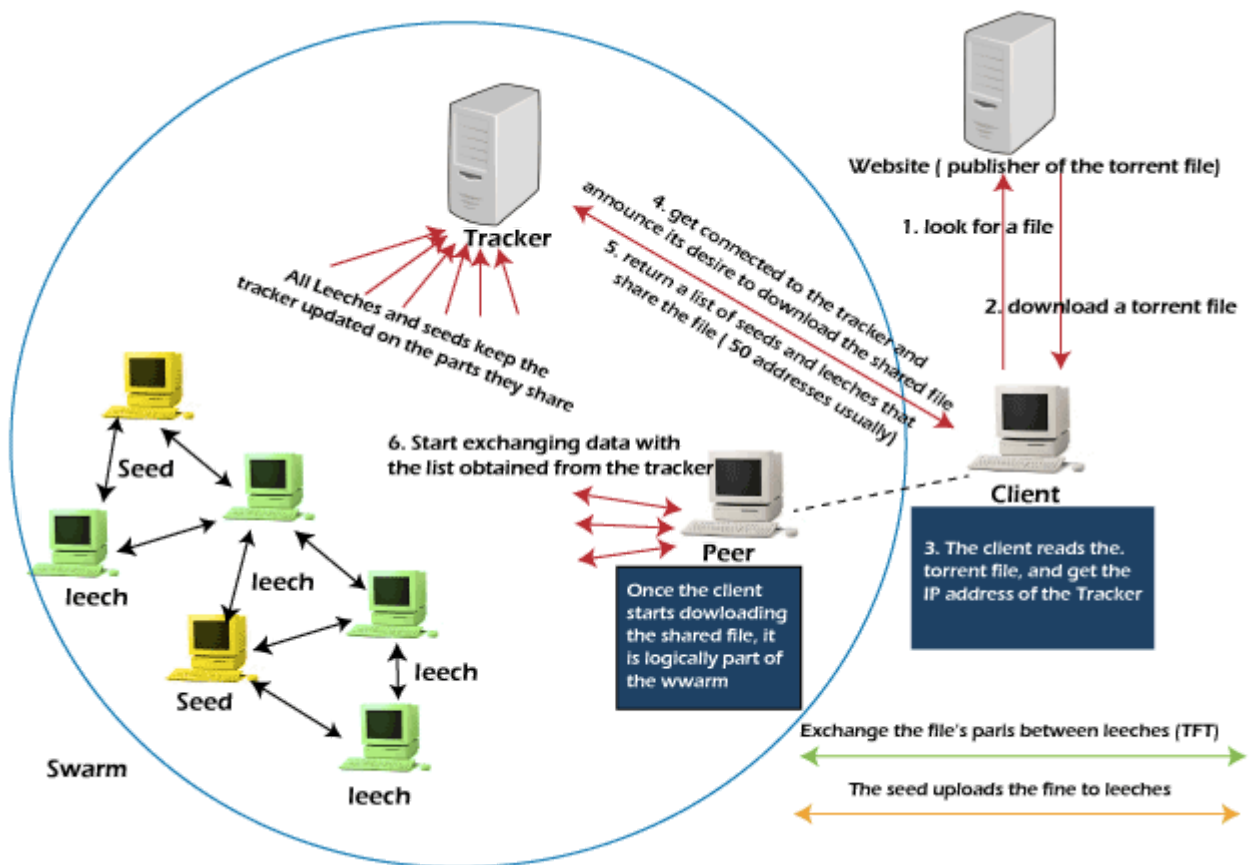
```

Tugas:

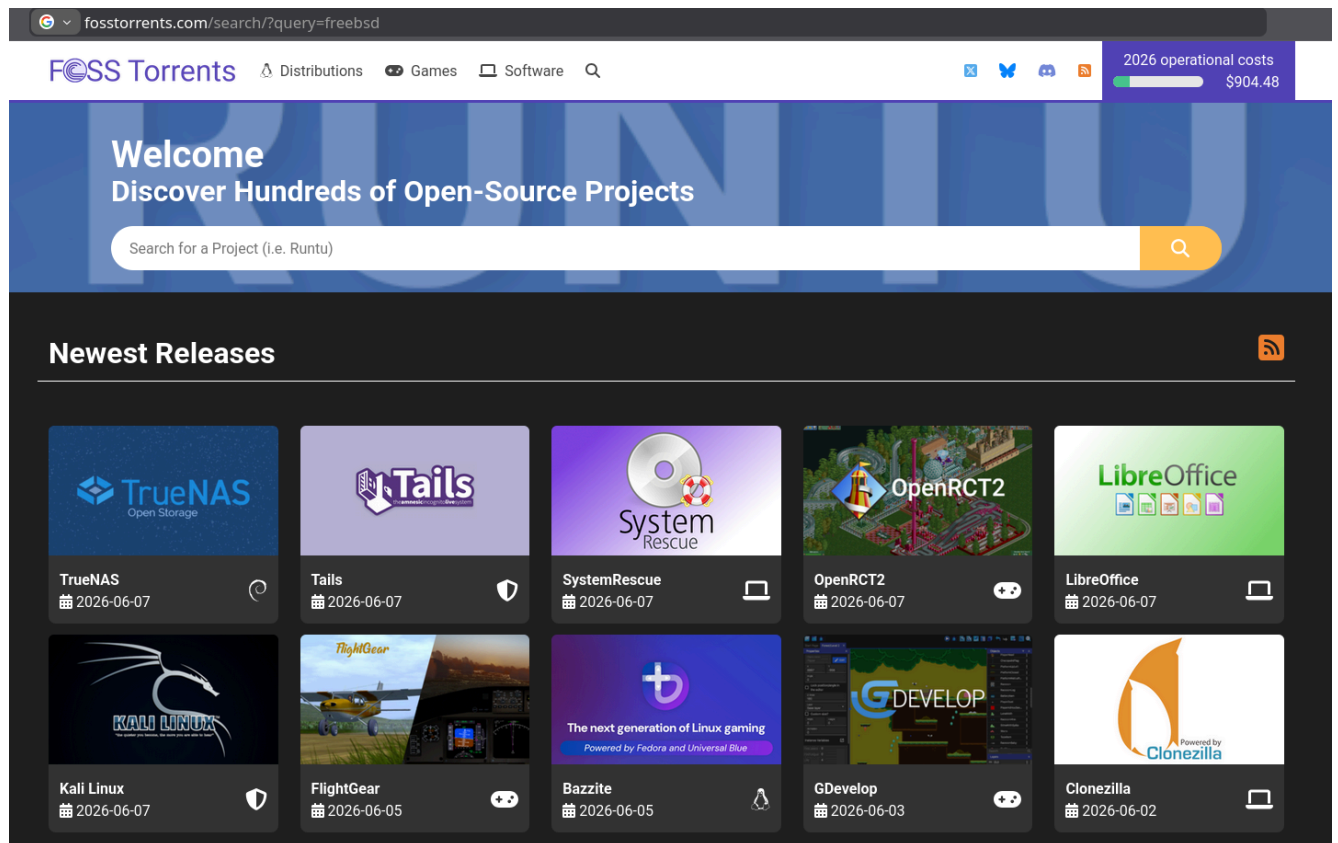
1. Jalankan program tersebut.
2. Jelaskan dengan singkat, apa yang dilakukan oleh program tersebut.
3. Dari program tersebut, jelaskan bagaimana DHT bisa digunakan untuk proses pencarian data yang berada pada node lainnya. Buat algoritmanya.

3. Torrent

Torrent adalah teknologi berbagi file *peer-to-peer* (P2P) yang memungkinkan pengguna mengunduh dan mengunggah file langsung antar perangkat tanpa melalui server pusat. Teknologi ini biasanya digunakan untuk mendistribusikan file berukuran sangat besar secara cepat dan efisien. Secara umum, berikut adalah cara kerja teknologi Torrent:



Salah satu *resources* di Web yang menyediakan torrent untuk berbagai ISO dari Linux dan / atau berbagai sistem operasi bebas serta software bebas lainnya adalah <https://fosstorrents.com/>.



File .torrent merupakan file yang berisi data tentang file yang ada di platform P2P BitTorrent. Berikut adalah contoh program Python untuk membaca data yang ada pada suatu file .torrent:

```
import bencoding
import hashlib

def baca_metadata_torrent(path_file_torrent):
    print(f"=== ANALISIS METADATA TORRENT: {path_file_torrent} ===")

    try:
        with open(path_file_torrent, 'rb') as f:
            data_torrent = bencoding.bdecode(f)

        print(f"[TRACKER URL] : {data_torrent.get('announce')}")

        info = data_torrent.get('info')
        if info:
            print(f"[NAMA FILE] : {info.get('name')}")
            print(f"[UKURAN FILE] : {info.get('length')} bytes")
            print(f"[UKURAN PIECE]: {info.get('piece length')} bytes")

            info_bencoded = bencoding.bencode(info)
            info_hash = hashlib.shal(info_bencoded).hexdigest()
            print(f"[INFO HASH] : {info_hash}")

            jumlah_pieces = len(info.get('pieces')) // 20
```



```

        print(f"[TOTAL PIECES]: {jumlah_pieces} potongan")

    except FileNotFoundError:
        print("Gagal: File .torrent tidak ditemukan. Pastikan path benar.")
    except Exception as e:
        print(f"Error saat membaca torrent: {e}")

if __name__ == "__main__":
    baca_metadata_torrent("FreeBSD-15.0-RELEASE-amd64-bootonly.iso.torrent")

```

```

1  import bcoding
1  import hashlib
2
3  def baca_metadata_torrent(path_file_torrent):
4      print(f"=== ANALISIS METADATA TORRENT: {path_file_torrent} ===")
5      ---
6      try:
7          with open(path_file_torrent, 'rb') as f:
8              data_torrent = bcoding.bdecode(f)
9              ---
10             print(f"[TRACKER URL] : {data_torrent.get('announce')}")
11             ---
12             info = data_torrent.get('info')
13             if info:
14                 print(f"[NAMA FILE] : {info.get('name')}")
15                 print(f"[UKURAN FILE] : {info.get('length')} bytes")
16                 print(f"[UKURAN PIECE]: {info.get('piece length')} bytes")
17                 ---
18                 info_bencoded = bcoding.bencode(info)
19                 info_hash = hashlib.sha1(info_bencoded).hexdigest()
20                 print(f"[INFO HASH] : {info_hash}")
21                 ---
22                 jumlah_pieces = len(info.get('pieces')) // 20
23                 print(f"[TOTAL PIECES]: {jumlah_pieces} potongan")
24             ---
25         except FileNotFoundError:
26             print("Gagal: File .torrent tidak ditemukan. Pastikan path benar.")
27         except Exception as e:
28             print(f"Error saat membaca torrent: {e}")
29
30 if __name__ == "__main__":
31     baca_metadata_torrent("FreeBSD-15.0-RELEASE-amd64-bootonly.iso.torrent")

```

INSERT ▶ main ▶ bdpd > read_torrent.py

Tugas:

1. Jalankan program tersebut.
2. Jelaskan mengapa program tersebut memberi output seperti saat dijalankan.
3. Ubahlah program tersebut sehingga lebih fleksibel: nama file .torrent yang akan dilihat metadata-nya menjadi argumen / parameter dari read_torrent.py. Contoh cara menjalankan:

```
python read_torrent.py namafile.torrent
```